



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第三章 软件建模



第二节 静态建模

概述

- 静态模型的概念

- 描述了被建模系统的静态结构，不随时间变化而变化
- 问题域静态建模是指分析阶段对系统的主要概念进行建模
- 问题域静态模型涉及：
 - 类
 - 类的属性
 - 类之间的关系以及约束
 - 识别软件外部类及上下文类图
 - 对象和类的组织



类之间的关系

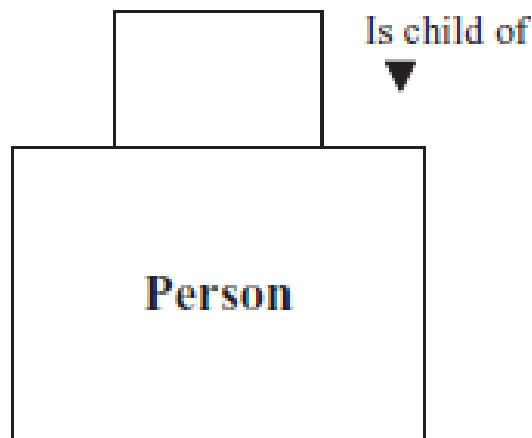
● 类之间的关联

- 关联定义了两个类之间的一种静态的、结构化的关系
- 类用名词；关联用动词或动词短语
- 关联的实例称为 **链接**
 - 两个对象之间可存在一个链接，当且仅当它们相应的类之间存在一个关联
- 关联的方向取决于相关类之间动作方向
- 关联可以是一元（自反）、二元或多元



类之间的关系

- 一元关联，也称自反关联。是一个类的一个对象与该类的另一个对象之间的关联

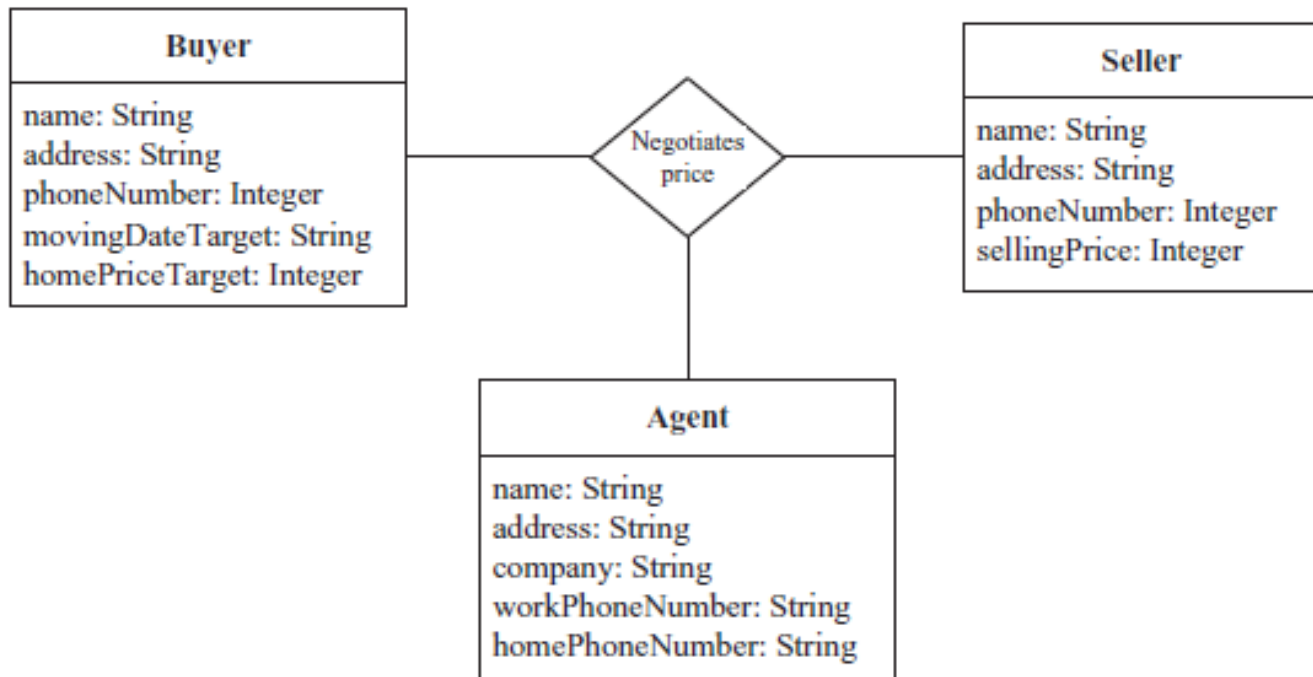


例如：“人”是“人”的孩子（Is child of）
“人”与“人”结婚（Is married to）
“雇员”是“雇员”的上司（Is leader of）



类之间的关系

- 三元关联。是在三个类间的关联

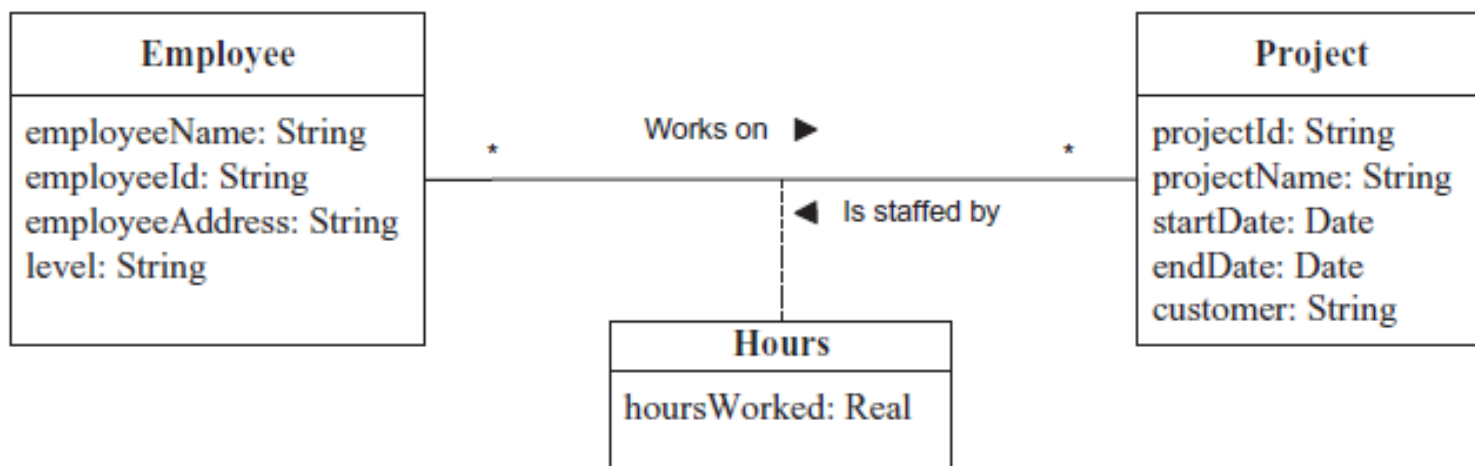


例如：“买方”通过“中介”和“卖方”协商价格



类之间的关系

- 关联类。对两个或多个类之间的关联进行建模的类



多对多关联类的例子：
“项目” 配备 “雇员”
“雇员” 工作于 “项目”

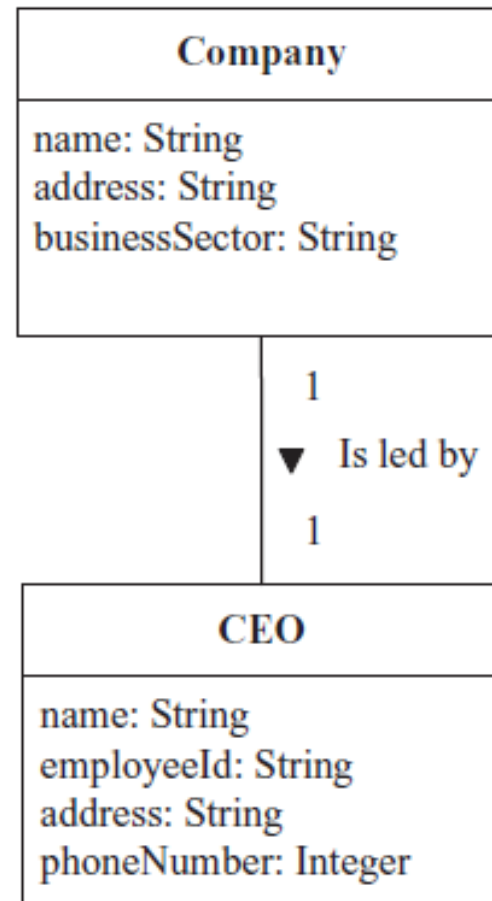


类之间的关系

○ 关联的多重性

● 一对一关联

- 任何一个类的一个对象只与另一个类的一个对象有一个链接

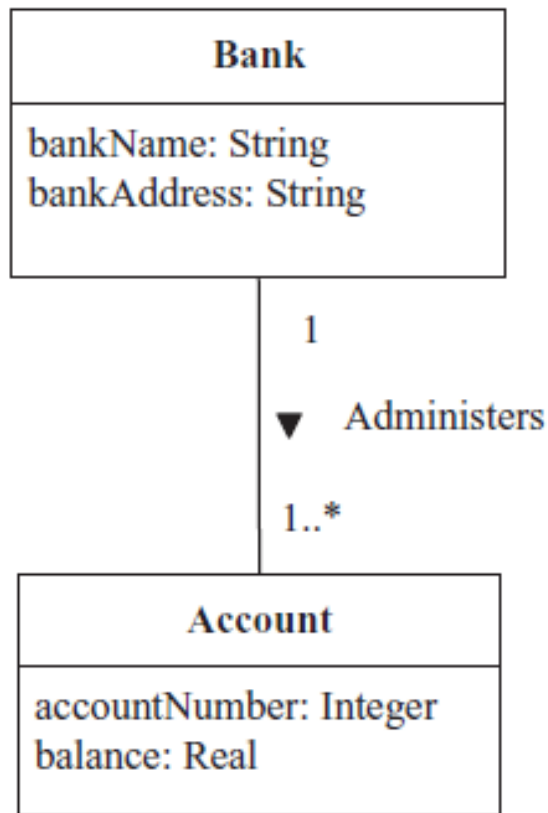


类之间的关系

○ 关联的多重性

● 一对多关联

- 两个类之间在一个方向上有一个一对多关联
- 在相反方向上是一个一对一关联

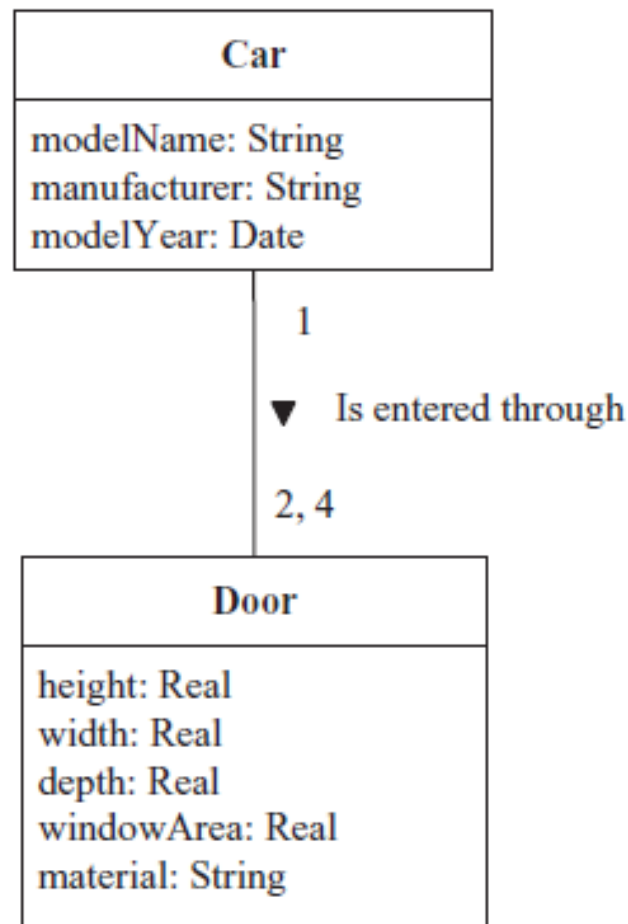


类之间的关系

○ 关联的多重性

● 规定数值的关联

- 指明了特定数字的关联

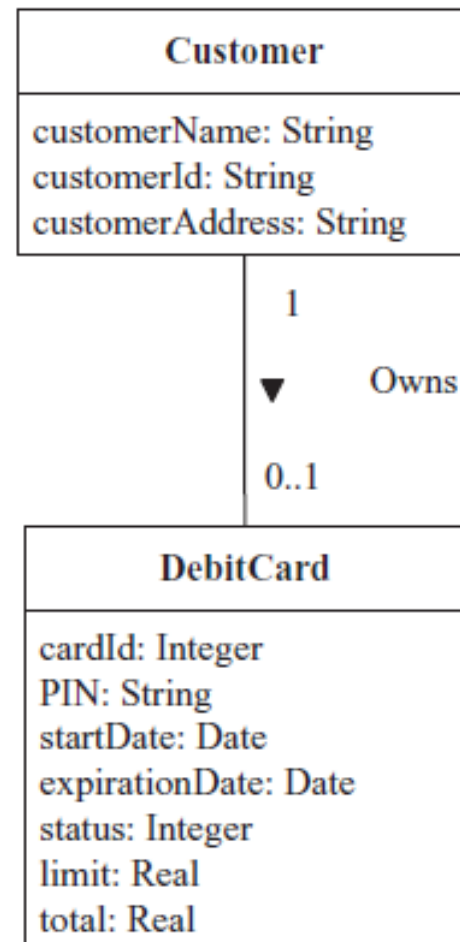


类之间的关系

○ 关联的多重性

● 可选关联

- 一个类的一个对象到另一个类的一个对象可能不总是存在链接

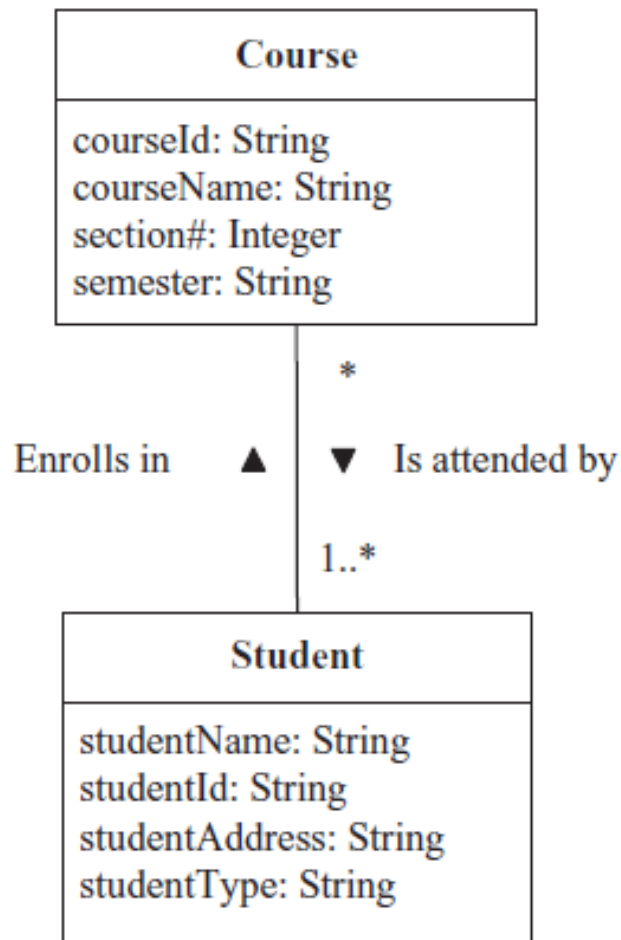


类之间的关系

○ 关联的多重性

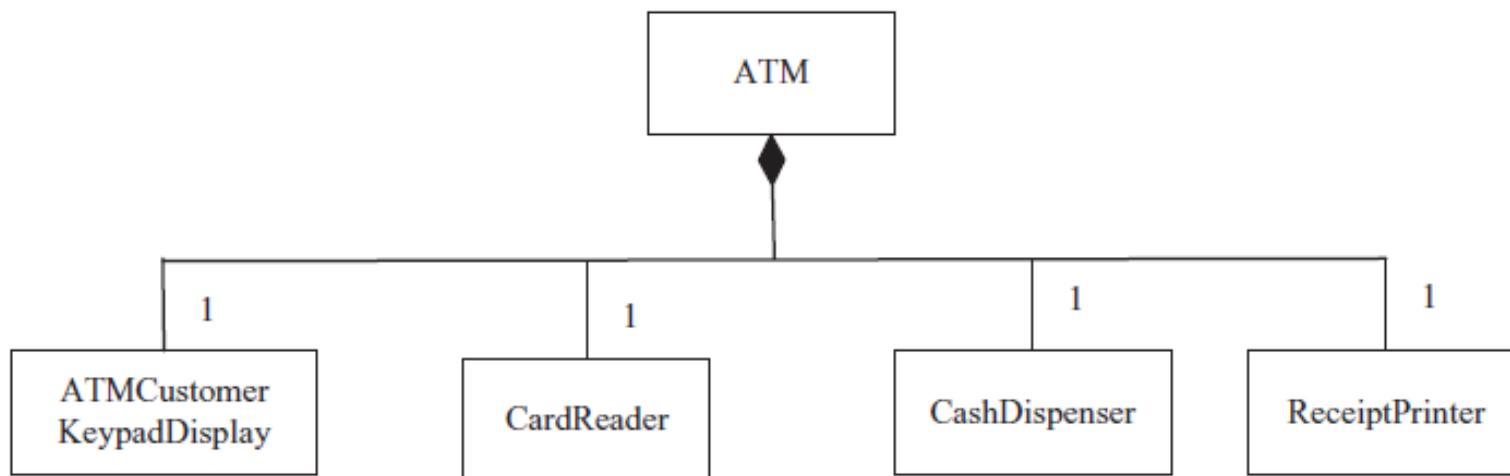
● 多对多关联

- 两个类之间的两个方向上各是一个一对多关联



类之间的关系

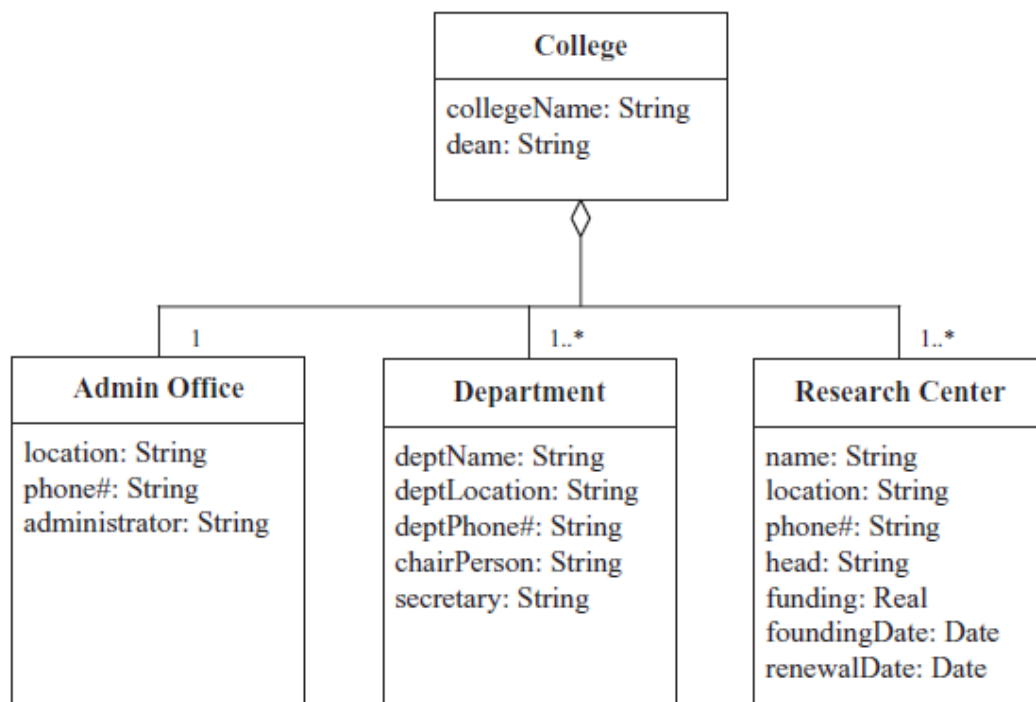
- 整体和部分（组合、聚合）关系
 - 组合比聚合的关系更强
 - 聚合比关联的关系更强
 - 组合类经常涉及整体和部分之间的物理关系



类之间的关系

○ 聚合层次是整体/部分关系的一种弱形式

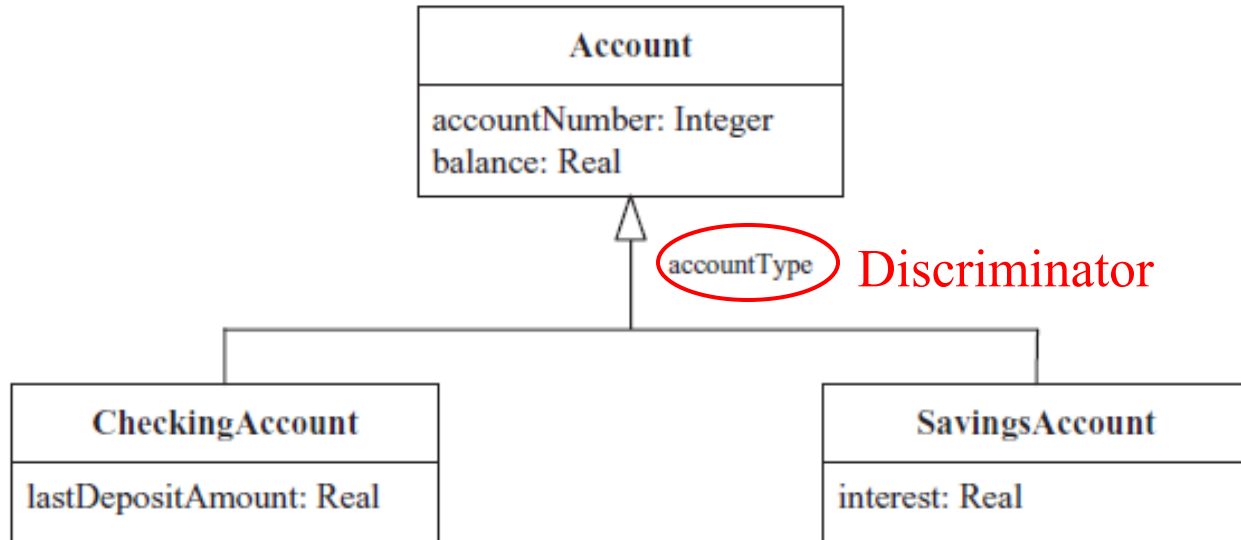
- 聚合里的部分实例能添加到聚合整体中，也能从聚合整体中移除
- 一般用于对概念
- 一个部分可以属于



类之间的关系

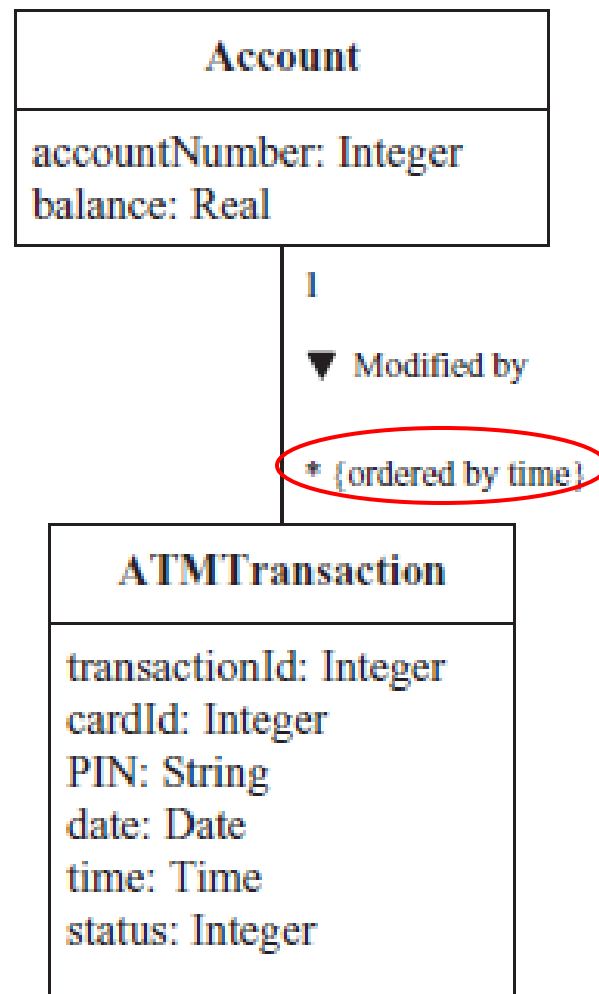
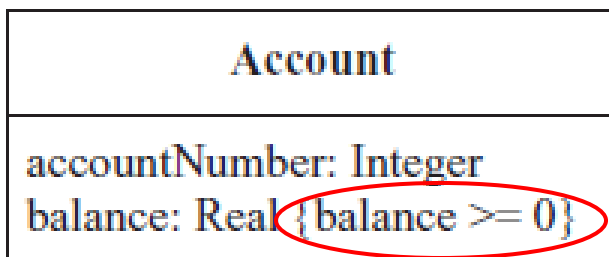
● 泛化/特化（继承）关系

- 类的共同属性被抽象到一个泛化类 **超类**
- 不同的属性表示了特化的性质 **子类**
- 超类和子类间是“Is a”的关系



约束

- 必须为真的条件或限制
- 对象约束语言 **OCL**
- 约束的使用方法
 - 对属性进行约束
 - 对关联的约束



问题域的静态建模——上下文类图

- 通过分析用例模型来对问题域建模
- 即对系统进行 **初始概念静态建模**
- 在设计阶段再实施更详细的静态建模
- 最初的重点：
 - 对 **物理类** 建模
 - 对 **实体类** 建模

具有物理特性的类
(物理设备、用户、
外部系统、计时器等)

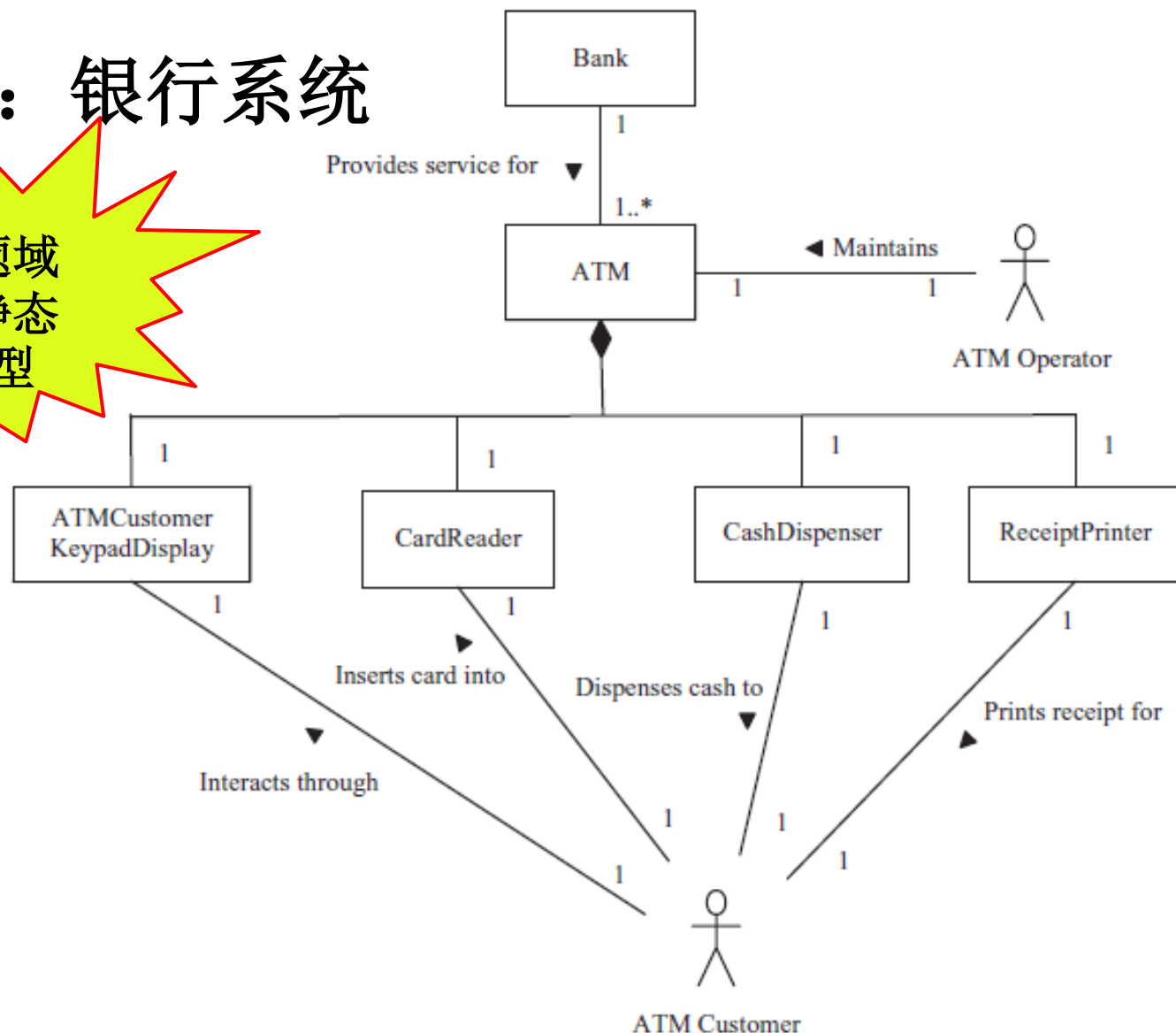
概念上的数据密集型类
(可持久化的信息实体)



问题域的静态建模—上下文类图

● 例子：银行系统

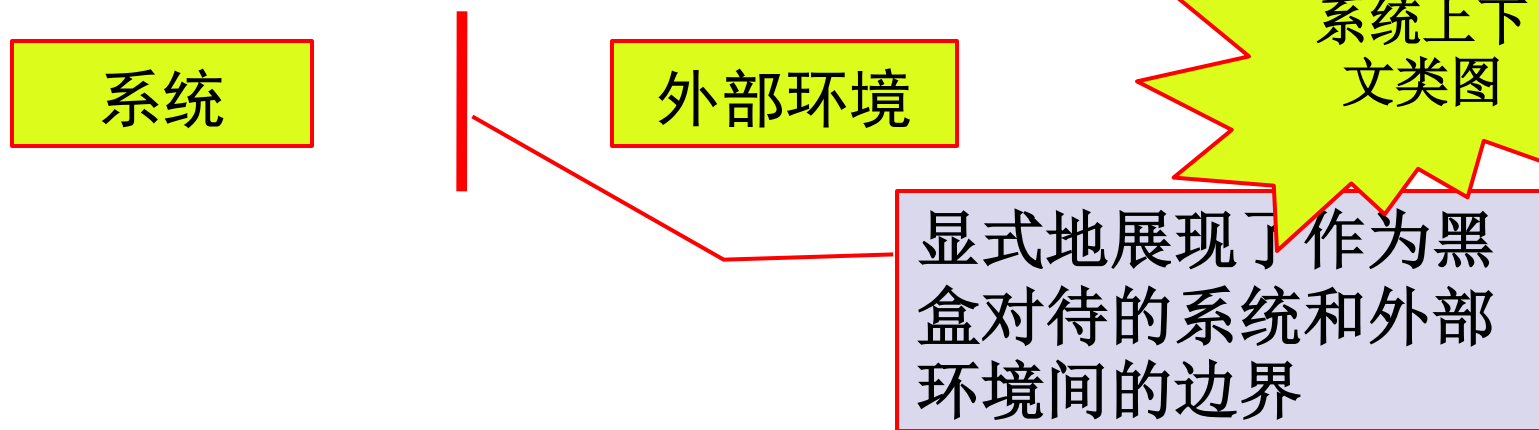
问题域的静态模型



问题域的静态建模—上下文类图

● 上下文的静态建模

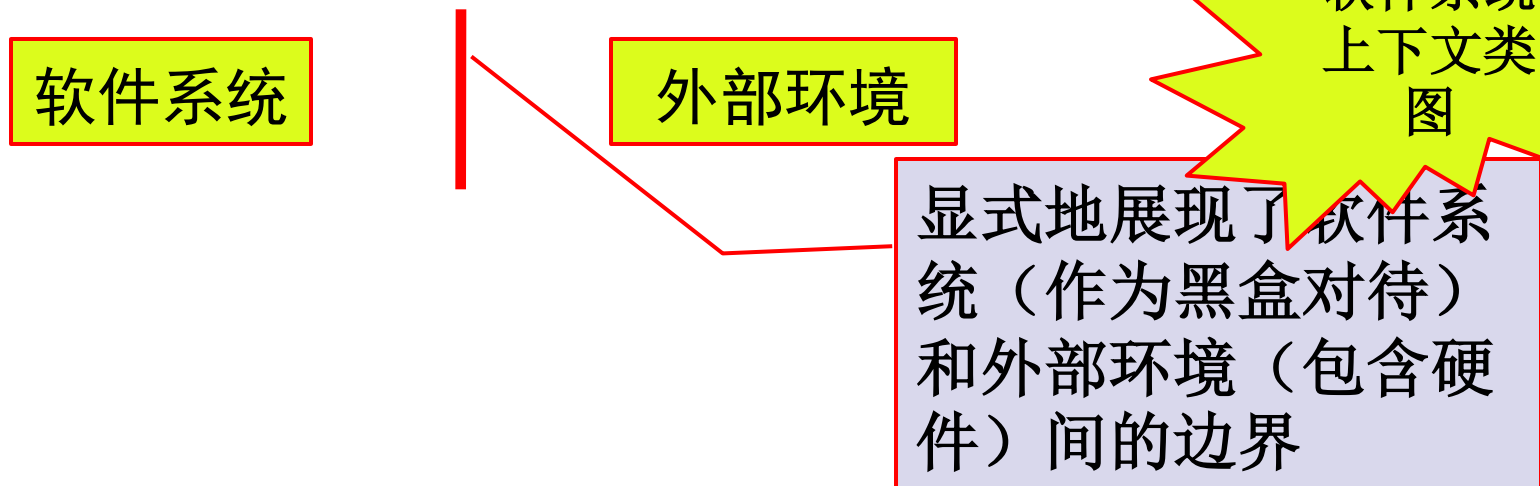
- 理解系统范围非常重要
- 上下文建模标识了系统内外部类
- 可以在整个系统（硬件和软件）级别建模
- 也可以在软件系统级别建模



问题域的静态建模—上下文类图

● 上下文的静态建模

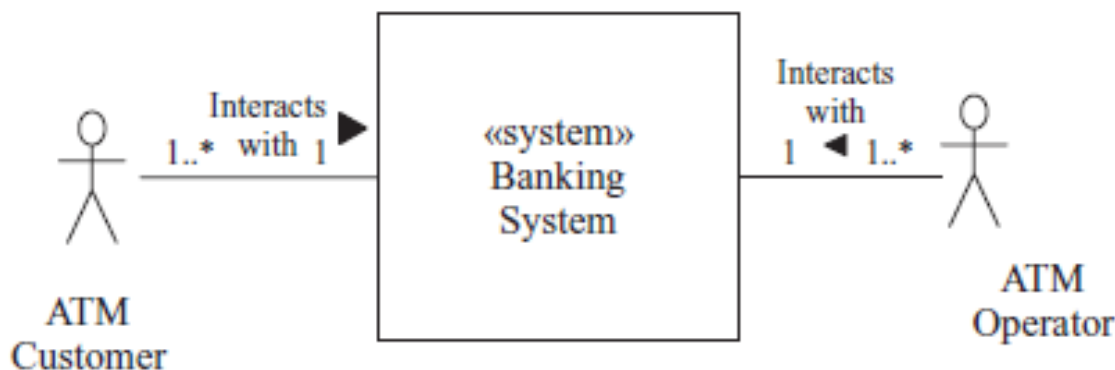
- 理解系统范围非常重要
- 上下文建模标识了系统内外部类
- 可以在整个系统（硬件和软件）级别建模
- 也可以在软件系统级别建模



问题域的静态建模—上下文类图

● 系统上下文类图

- 在考虑软件系统上下文类图之前建模
- 只有人类参与者和外部系统在系统边界之外
- I/O设备是系统硬件的一部分，属于系统内部类
- 如：银行系统的上下文类图

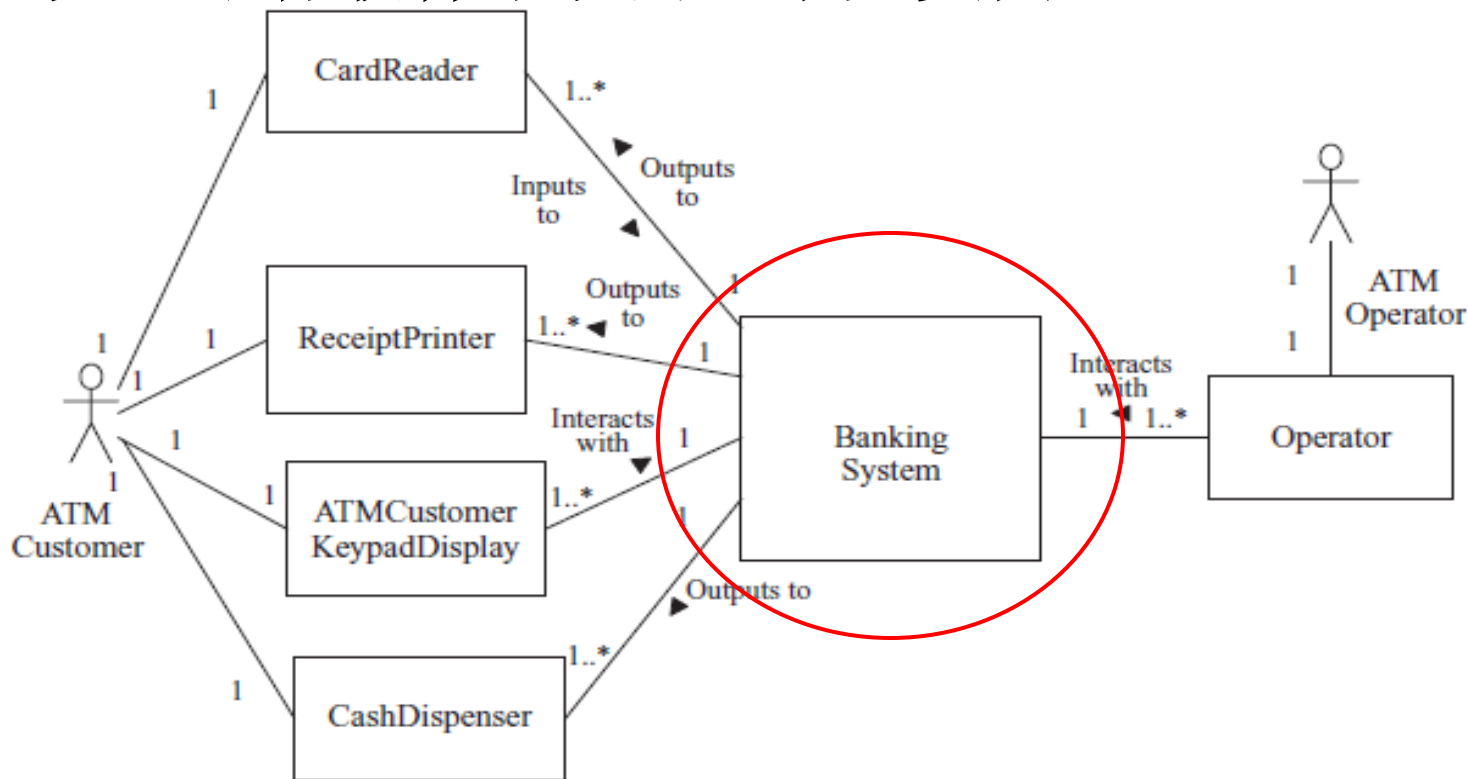


问题域的静态建模——上下文类图

- 软件系统上下文类图

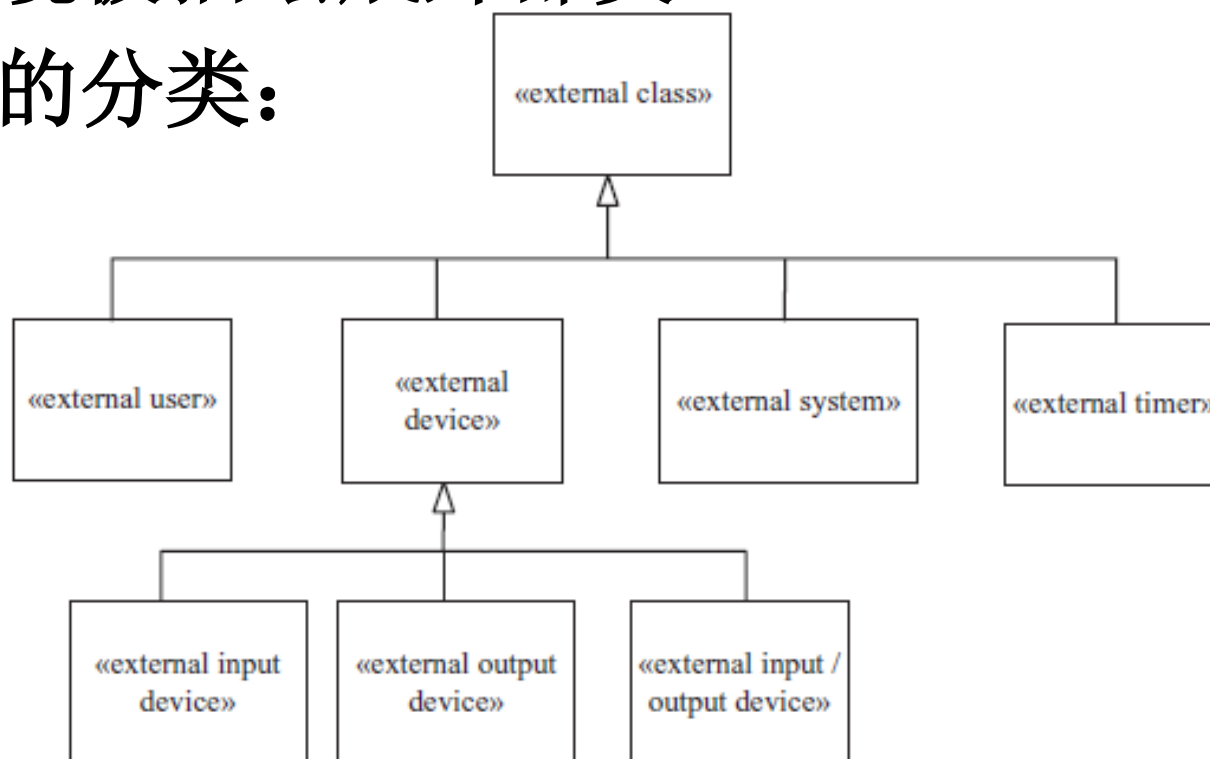
- I/O设备属于系统外部类

- 如：银行软件系统的上下文类图



识别外部类

- 将硬件/软件系统显示为一个聚合类
 - 用<<system>>构造型表示
- 外部环境被描绘成外部类
- 外部类的分类:



识别外部类

- 通过标准输入/输出设备和软件系统交互的外部用户描述为 **外部用户** <<external user>>
 - 标准输入/输出设备由操作系统处理
 - 我们对用户接口更感兴趣
 - 不把标准输入/输出设备建模为外部类
 - 将非标准输入/输出设备建模为外部类



识别外部类

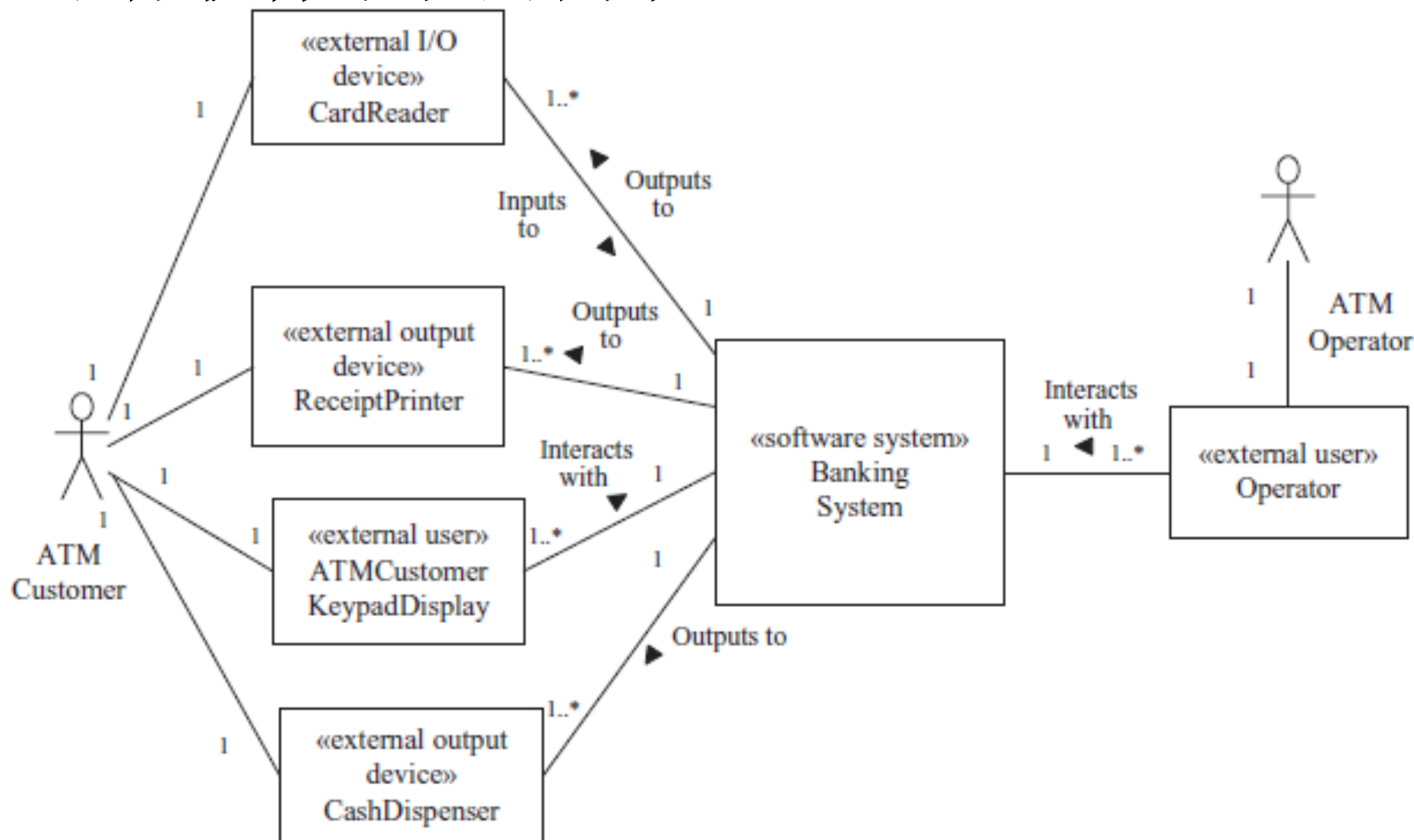
- 参与者与外部类

- 参与者是一个比外部类更抽象的概念
- 输入/输出设备参与者 → 外部输入/输出设备类
- 外部系统参与者 → 外部系统类
- 计时器参与者 → 外部计时器类
- 人类参与者
 - 通过标准输入/输出设备与系统交互，建模为人类参与者的名称的类
 - 通过非标准输入/输出设备与系统交互，建模为人类参与者通过外部输入/输出设备类与系统交互



识别外部类

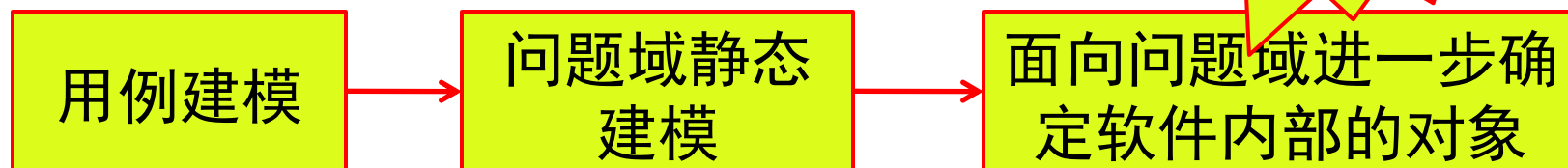
● 银行软件系统的例子：



对象和类的组织

- 确定软件系统内部的软件对象

仍是问题
域对象

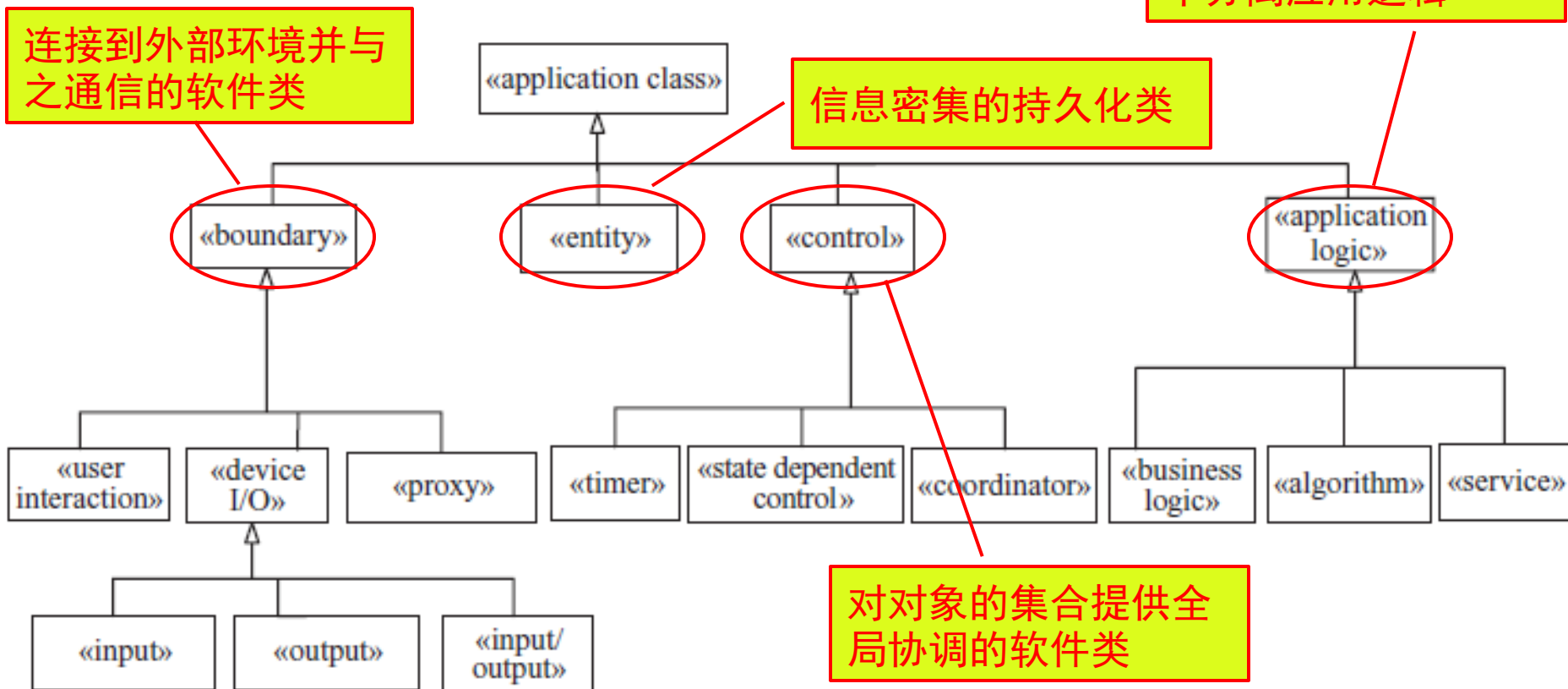


- 在问题域中寻找真实世界对象，然后设计相对应的软件对象对真实世界建模
- 软件对象实现了每一个用例



对象和类的组织

● 用构造型对软件对象/类分类



对象和类的组织

● 实体类和对象

○ 实体类是概念性的数据密集型类

- 存储数据并提供对这些数据的访问
- 能够被持久化

○ 实体类模型类似于逻辑数据库模式

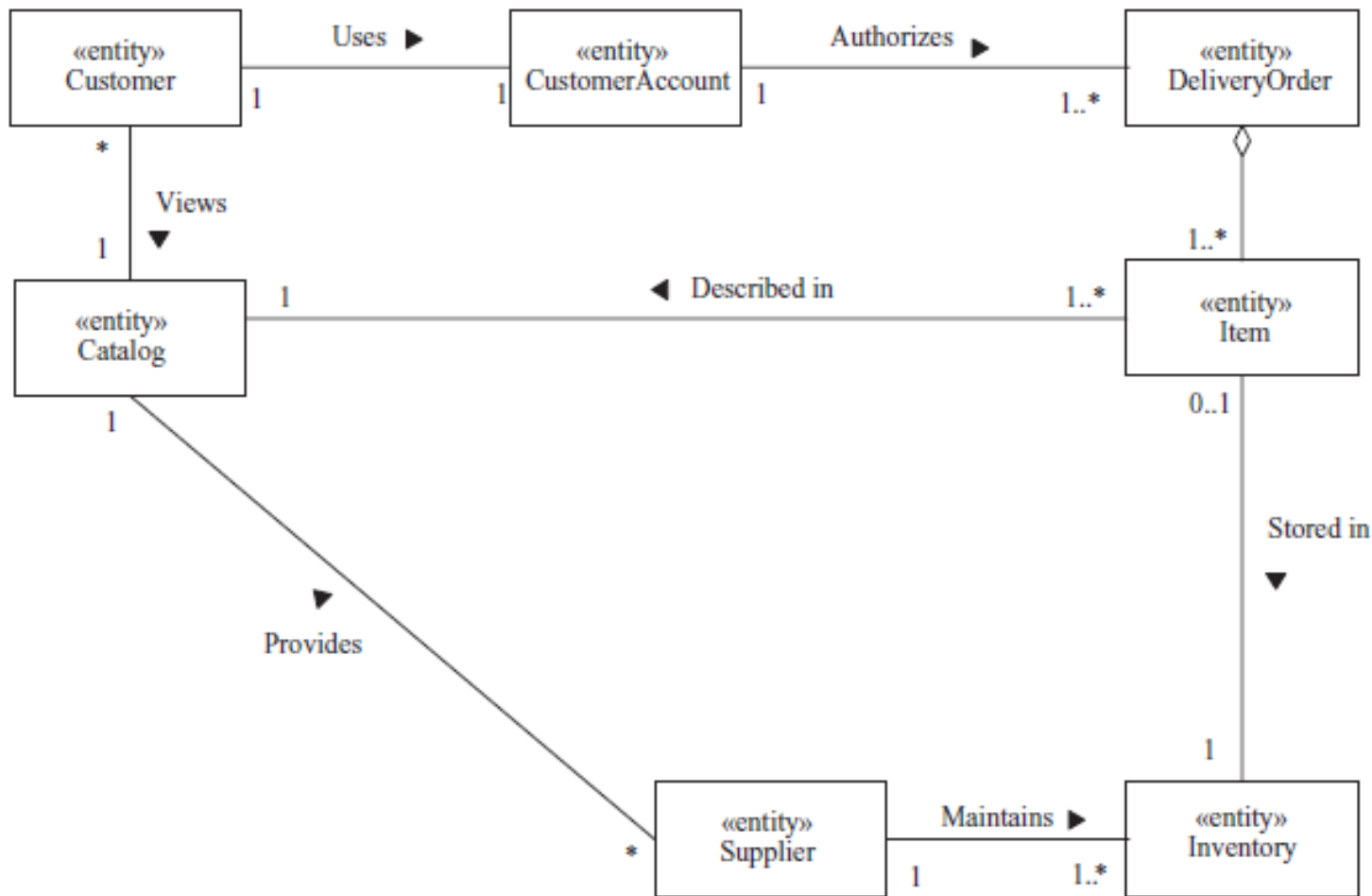
● 实体类图——ER图

- 不同之处：实体类可以有操作，但是在问题域静态建模阶段，重点仍是定义实体类、它们的属性和它们间的关系，操作的定义推迟到在设计建模



对象和类的组织

○ 在线购物软件的例子：



对象和类的组织

○ 在线购物软件的例子：

● 部分实体类属性

«entity» DeliveryOrder
orderId : Integer orderStatus : OrderstatusType accountId : Integer amountDue: Real supplierId : Integer creationDate : Date plannedShipDate : Date actualShipDate : Date paymentDate: Date

«entity» Customer
customerId : Integer customerName : String address : String telephoneNumber : String faxNumber : String email : EmailType

«entity» Inventory
itemID : Integer itemDescription : String quantity : Integer price : Real reorderTime : Date

«entity» Catalog
itemId : Integer itemDescription : String unitCost : Real supplierId : Integer itemDetails : linkType



对象和类的组织

● 边界类和对象

○ 外部类与软件边界类的对应

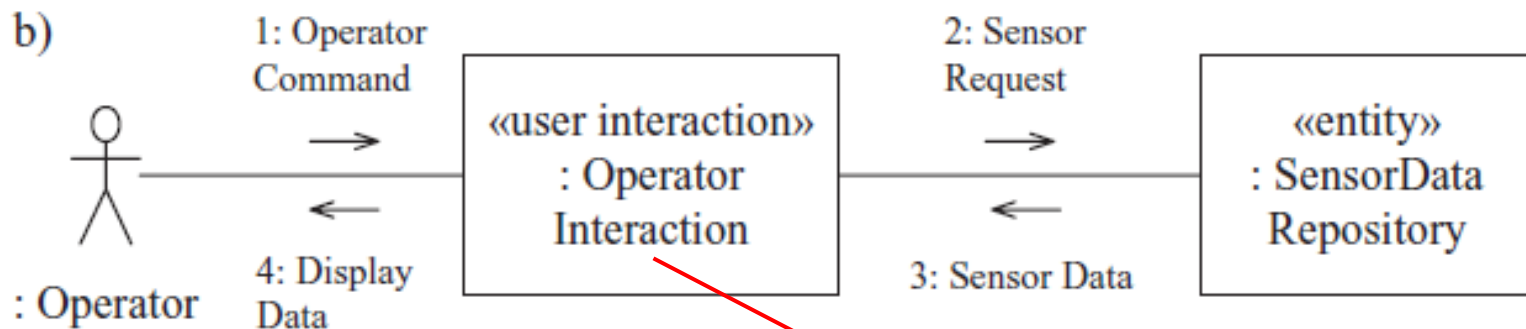
- 边界类是软件内部的类，通过接口连接到外部类并与之通信
- 每一个外部类都与系统中的边界类进行通信
- 对应关系：
 - 外部用户类 —————> 用户交互类
 - 外部系统类 —————> 代理类
 - 外部设备类 —————> 设备I/O边界类
 - 外部输入设备类 —————> 输入类
 - 外部输出设备类 —————> 输出类
 - 外部I/O设备类 —————> I/O类
 - 外部计时器类 —————> 软件计时器类



对象和类的组织

○ 用户交互对象

- 人类用户与它通信
- 依赖于用户接口技术
 - 如，命令行接口；图形用户界面（**GUI**）等
- 可以是组合对象，由简单的用户交互对象组成
- 例子：



可以是组合的用户交互对象

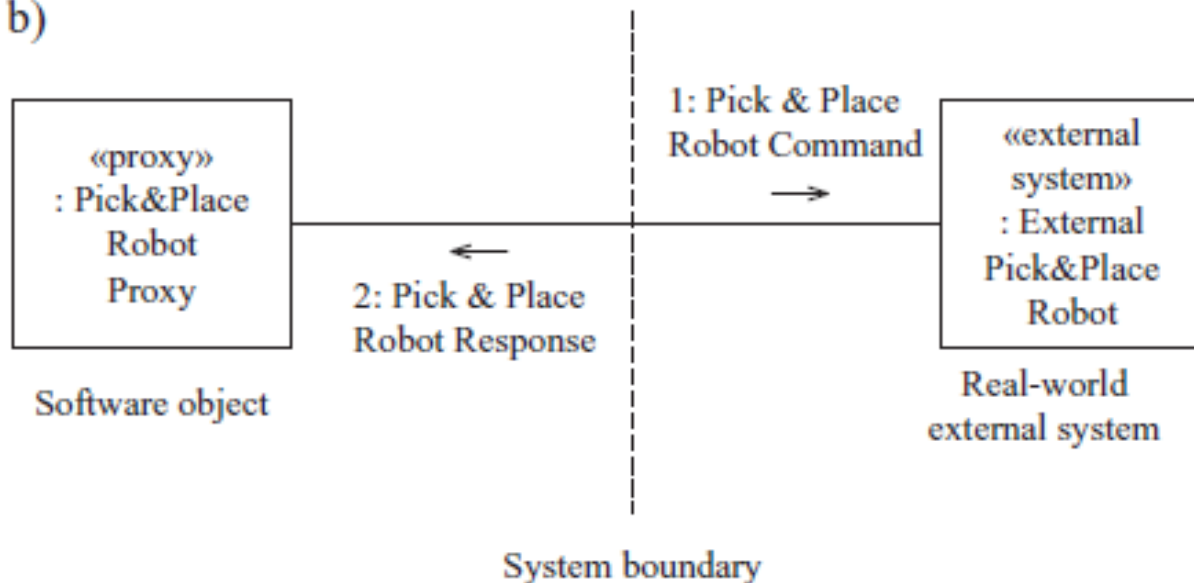


对象和类的组织

○代理对象

- 通过接口连接到外部系统并与之通信
- 是外部系统的本地代表
- 代理对象隐藏了“如何”与外部系统通信的细节
- 例子：

b)



对象和类的组织

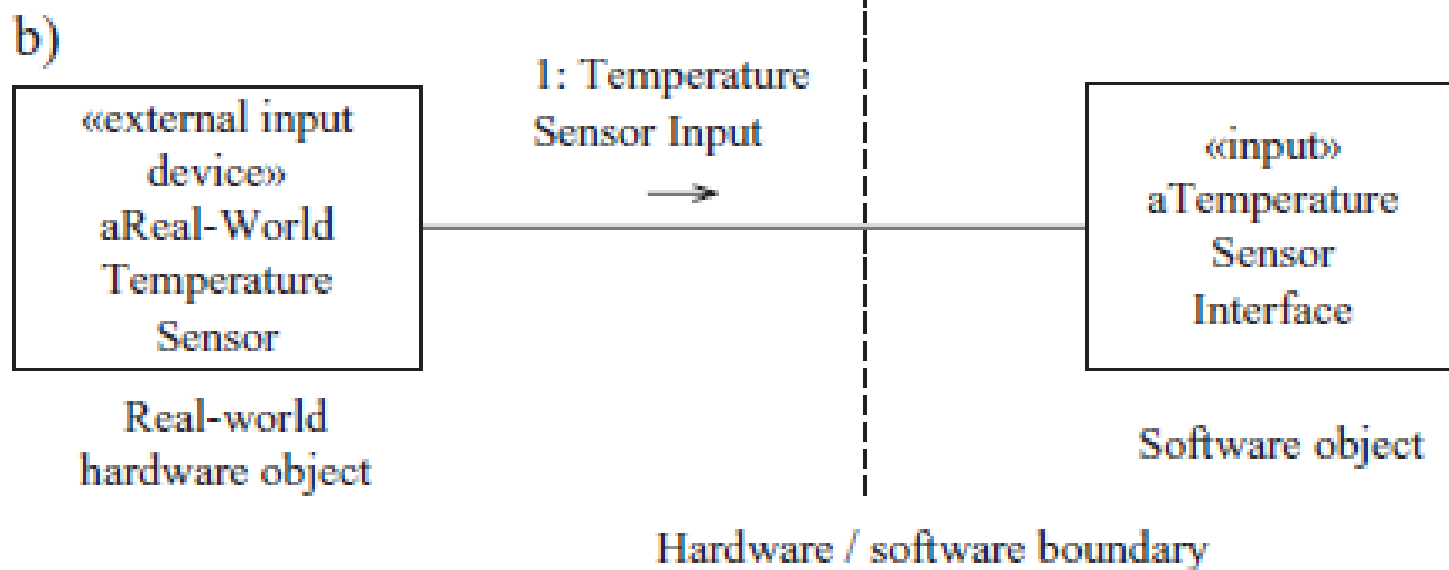
○ 设备I/O边界对象

- 对一个硬件I/O设备提供软件接口
- 主要针对非标准的I/O设备
- 每一个 **与问题相关的** 真实世界对象，在系统中都要有与之相对应的软件对象
- **与问题相关的** 真实世界对象，通常通过传感器和执行器连接到系统
 - 通过 **传感器** 向系统提供输入
 - 通过 **执行器** 接收来自系统的输出
- 分为：
 - 输入对象
 - 输出对象
 - 输入/输出对象



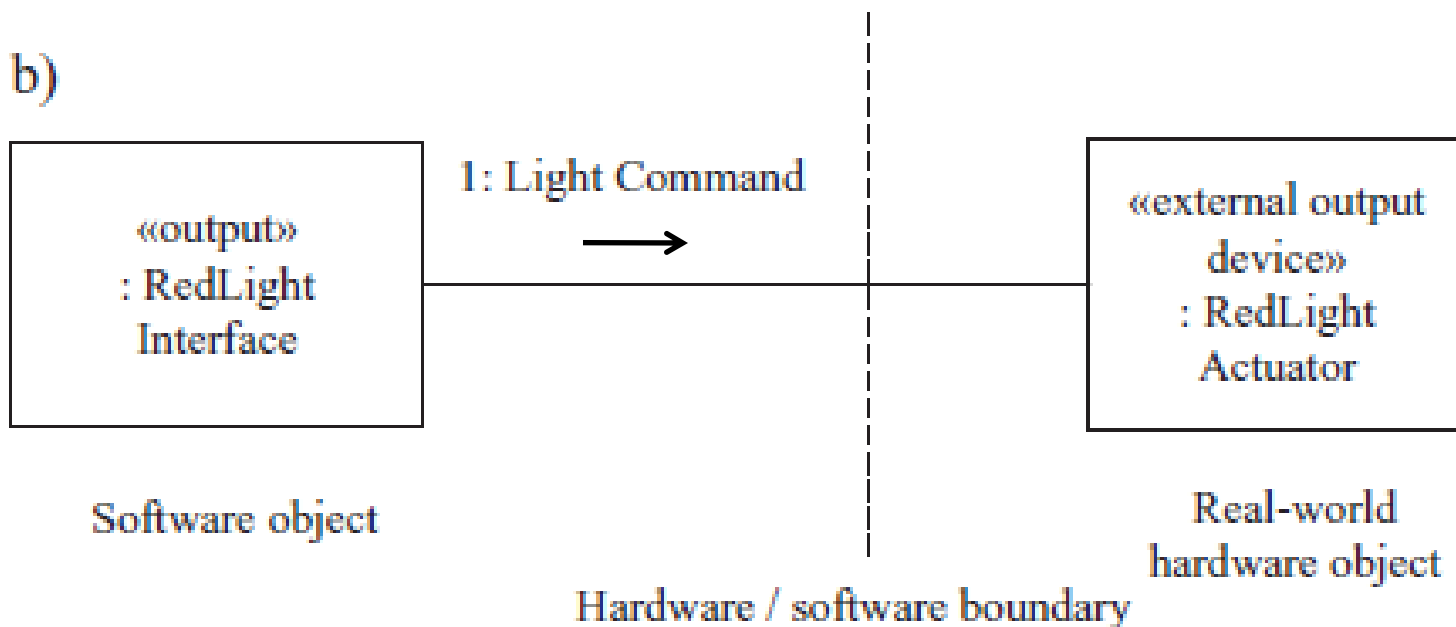
对象和类的组织

- 例子：输入对象



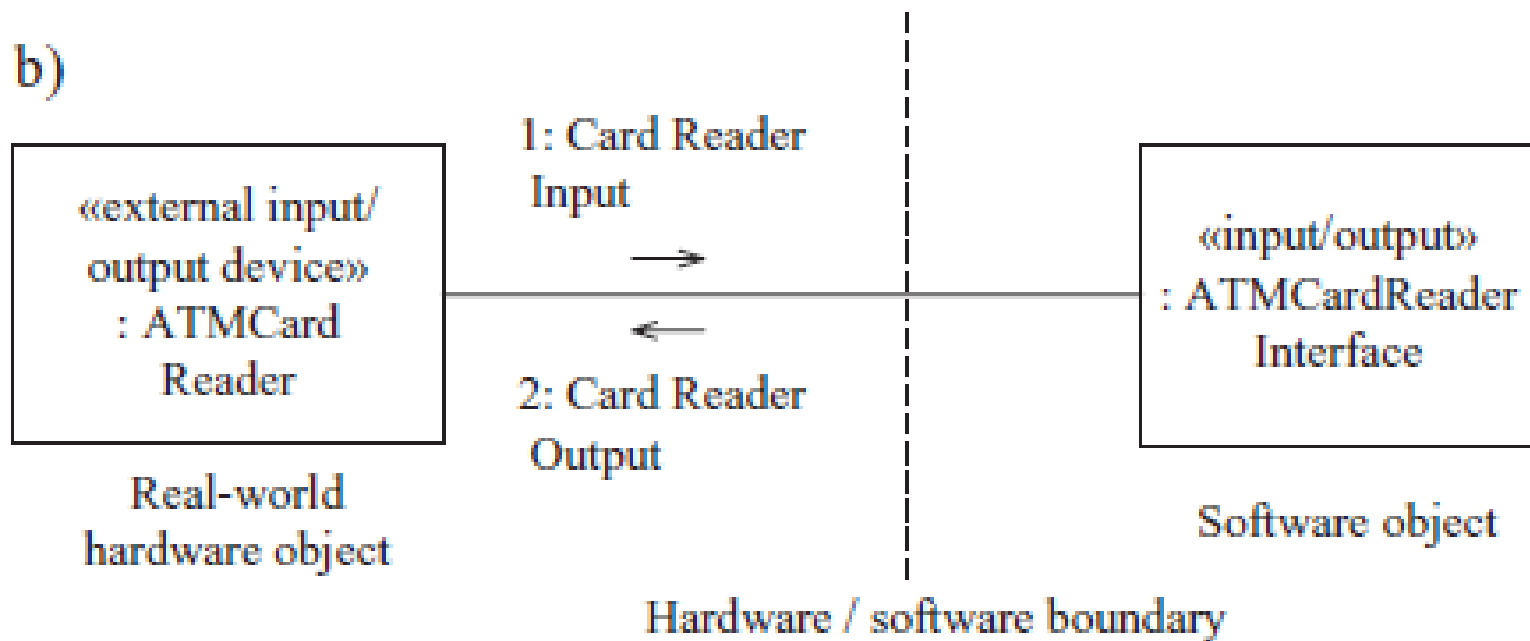
对象和类的组织

- 例子：输出对象



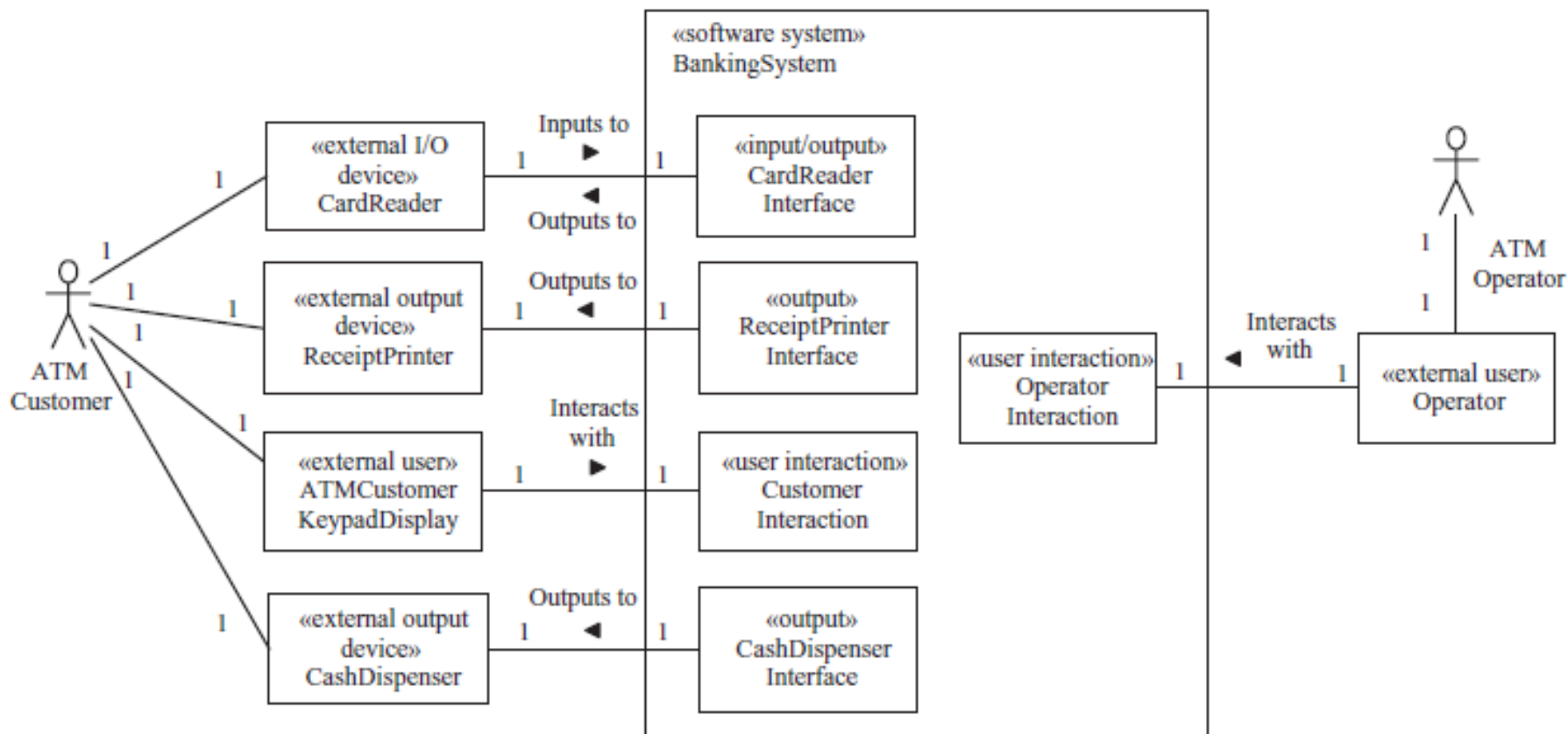
对象和类的组织

- 例子：输入/输出（I/O）对象

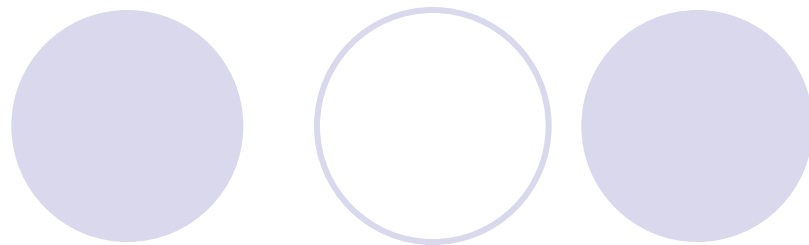


对象和类的组织

外部类与边界类的通信例子



对象和类的组织



● 控制类和对象

- 用于对实现用例的对象进行协调
- 一般用于复杂用例中
- 它控制参与用例的其他对象的行为，通知这些对象什么时候做什么
- 控制对象可以是状态相关的
- 分为：
 - 协调者对象
 - 状态相关的控制对象
 - 计时器对象



对象和类的组织

○协调者对象

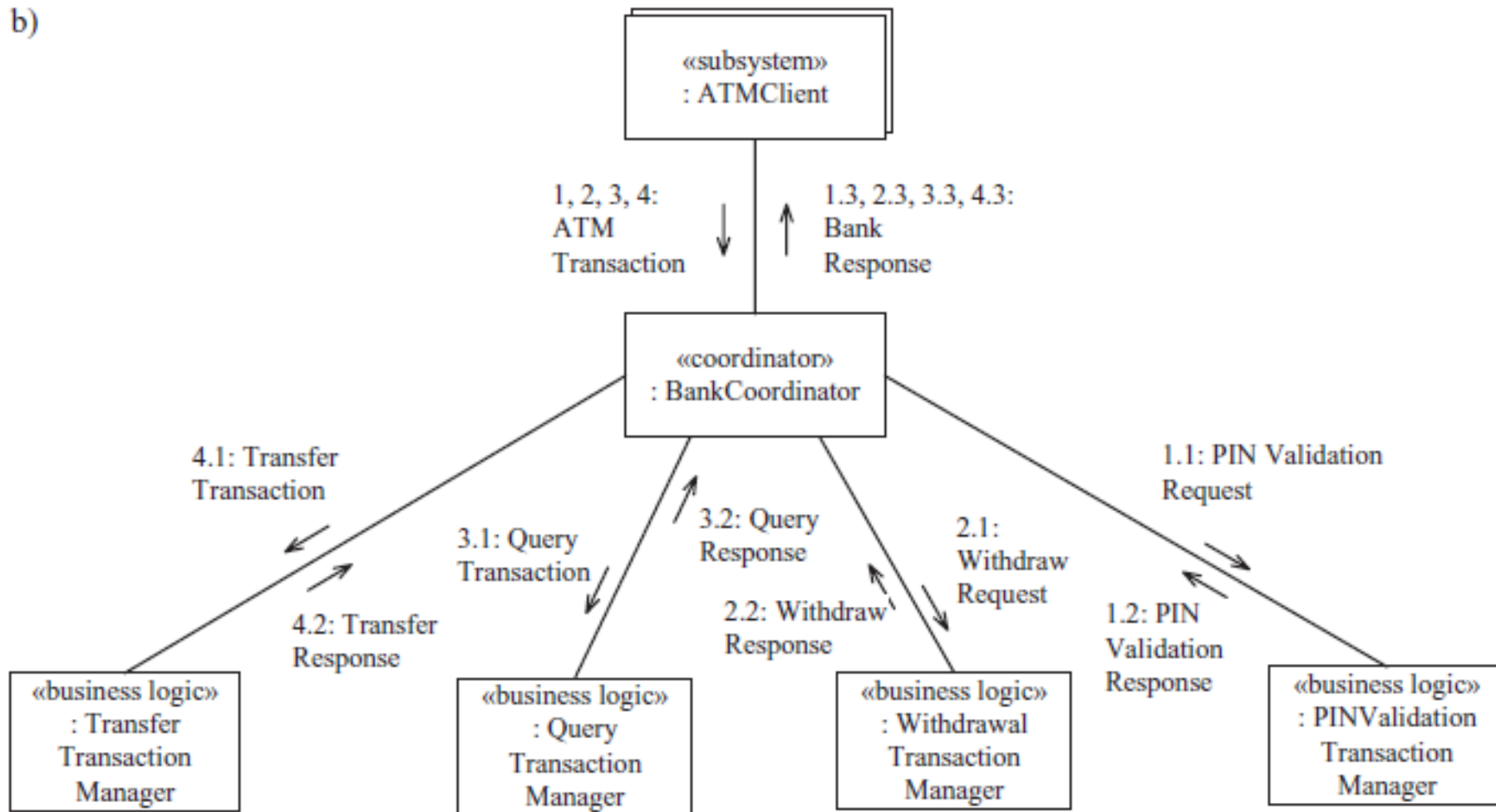
- 是做出总体决策的对象，它确定相关对象的协作顺序，从而为用例的执行提供总体顺序安排
- 决定了其他对象何时、以何种顺序参与到用例中
- 根据接收到的输入做出决策，并且与状态无关
- 即：由协调者对象发起的动作只取决于包含在传入消息中的信息，而不依赖于系统中之前发生的事情



对象和类的组织

- 协调者对象的例子：银行协调者对象

b)



对象和类的组织

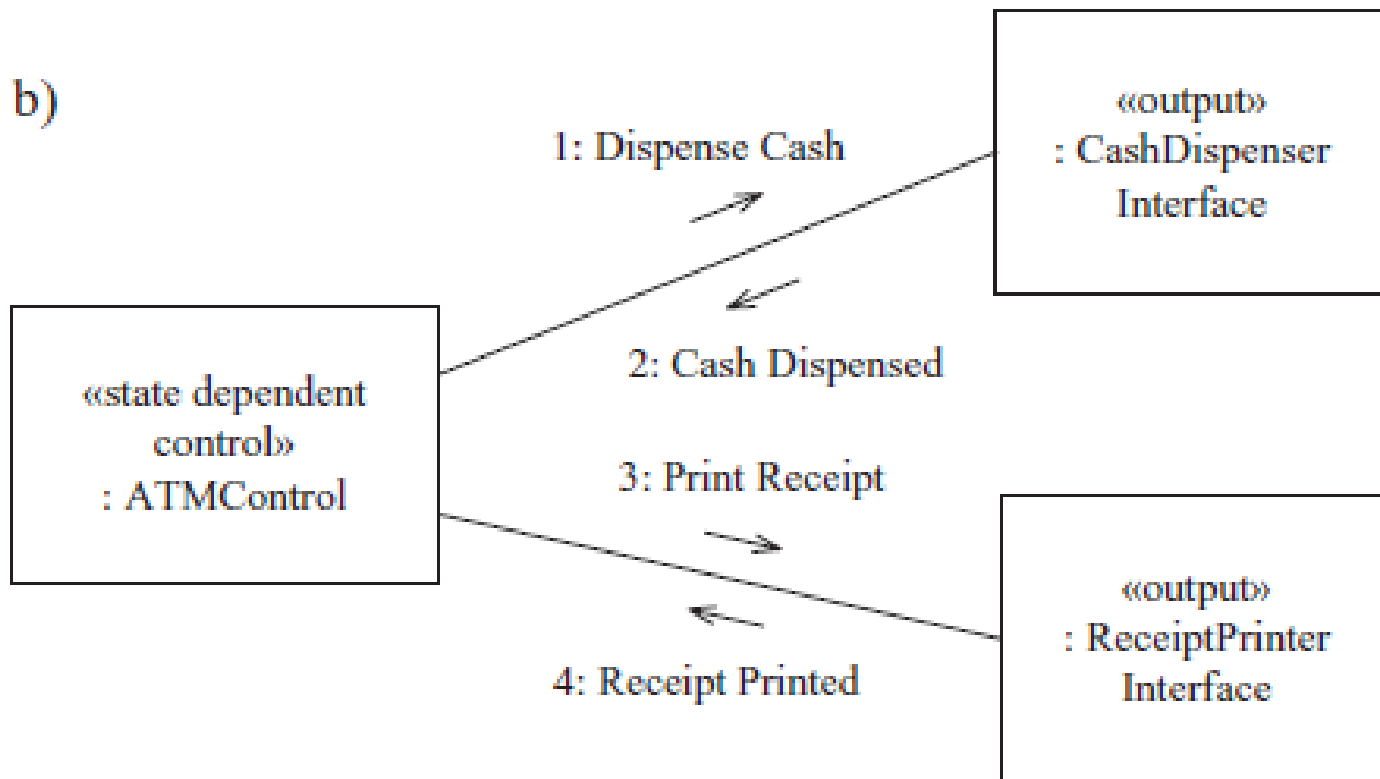
○ 状态相关的控制对象

- 在不同状态下其行为有变化
- 用有限状态机对它进行定义，用状态图来描述
- 接收引起状态转移的输入事件，并产生控制其它对象的输出事件
- 即： *由状态相关的控制对象产生的输出事件不仅依赖该对象的输入，还依赖对象的当前状态*



对象和类的组织

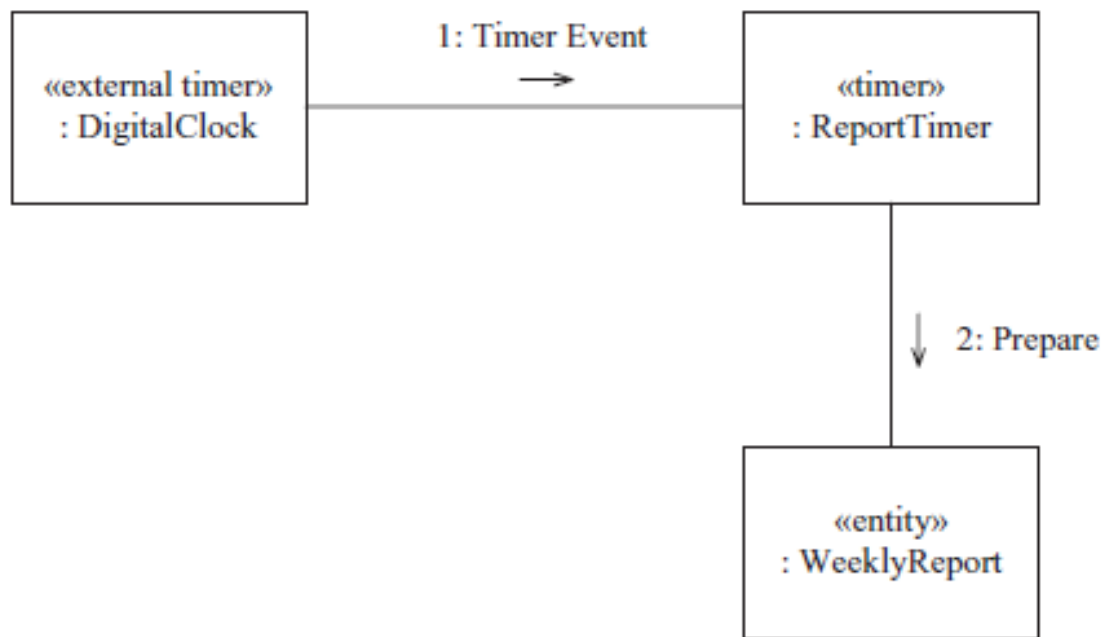
- 状态相关的控制对象的例子：**ATM控制**



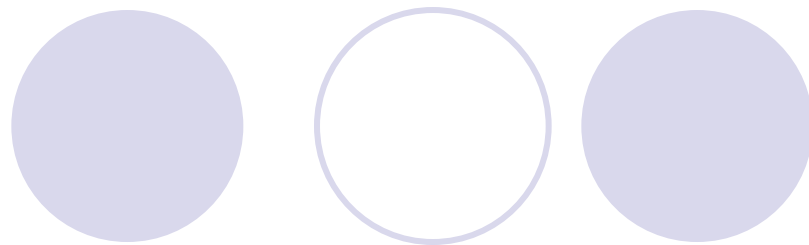
对象和类的组织

○ 计时器对象

- 由外部计时器（如实时时钟或操作系统时钟）激活的控制对象
- 要么自己执行某个动作，要么激活另一个对象来执行期望的动作



对象和类的组织



● 应用逻辑类和对象

- 应用逻辑对象和控制对象都更可能是在开发动态模型时考虑，但在开发静态模型后期就要涉及
- 分成：
 - 业务逻辑对象
 - 算法对象
 - 服务对象



对象和类的组织

○ 业务逻辑对象

● 处理应用逻辑

● 目标：

- 将可能相互独立变化的业务规则封装到分离的业务逻辑对象中
- 将业务规则从它们操作的实体数据中分离出来，使得业务逻辑可以独立于实体数据而变化

● 业务逻辑对象在执行期内访问各种实体对象

● 设计决策：

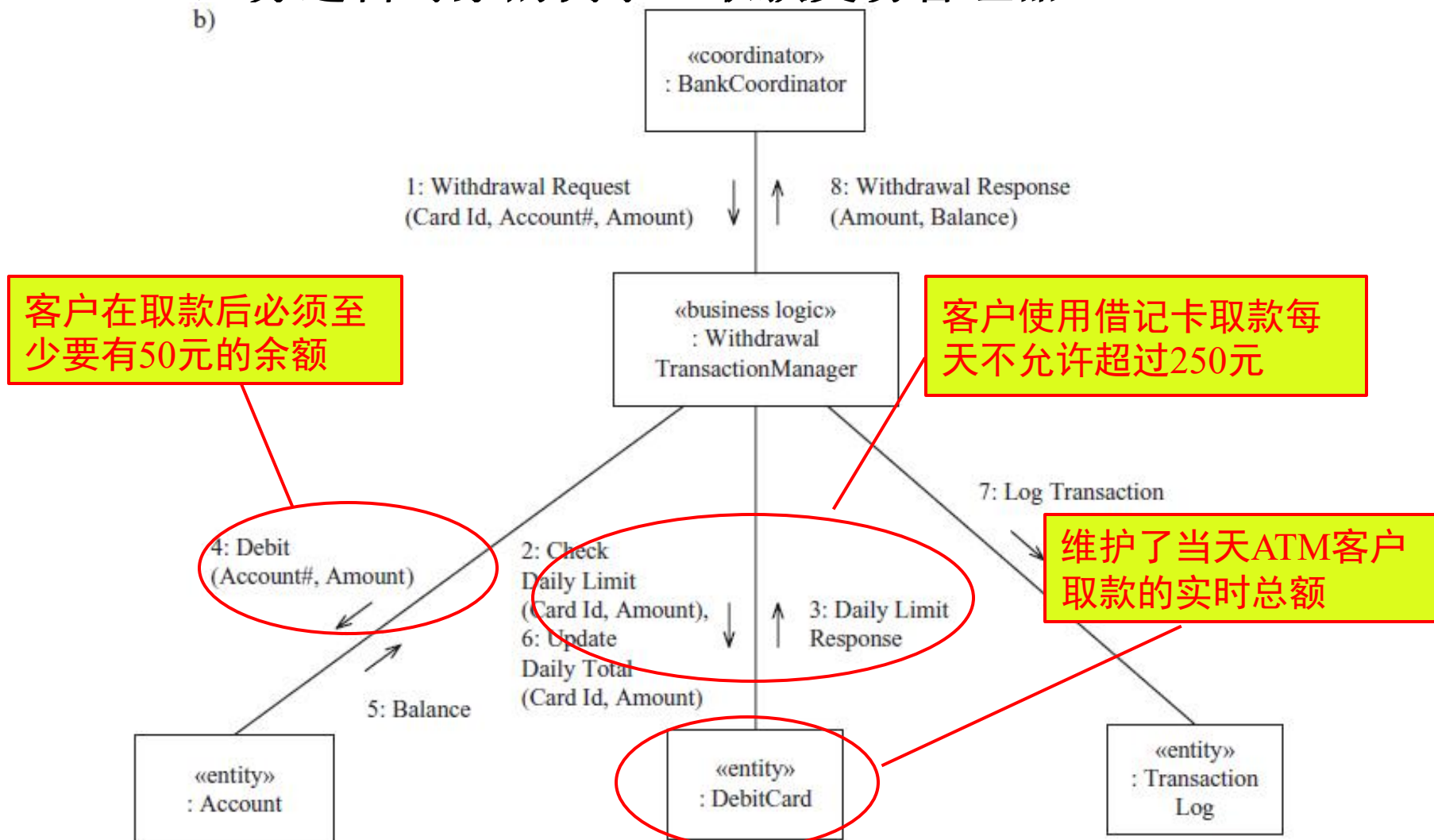
- 如果业务规则只有通过访问 **两个或更多** 的实体对象才能执行，就应该有一个分离的业务逻辑对象
- 如果访问 **一个** 实体对象就足以执行业务规则，则可将它作为该对象的一个操作



对象和类的组织

业务逻辑对象的例子：取款交易管理器

b)



对象和类的组织

○ 算法对象

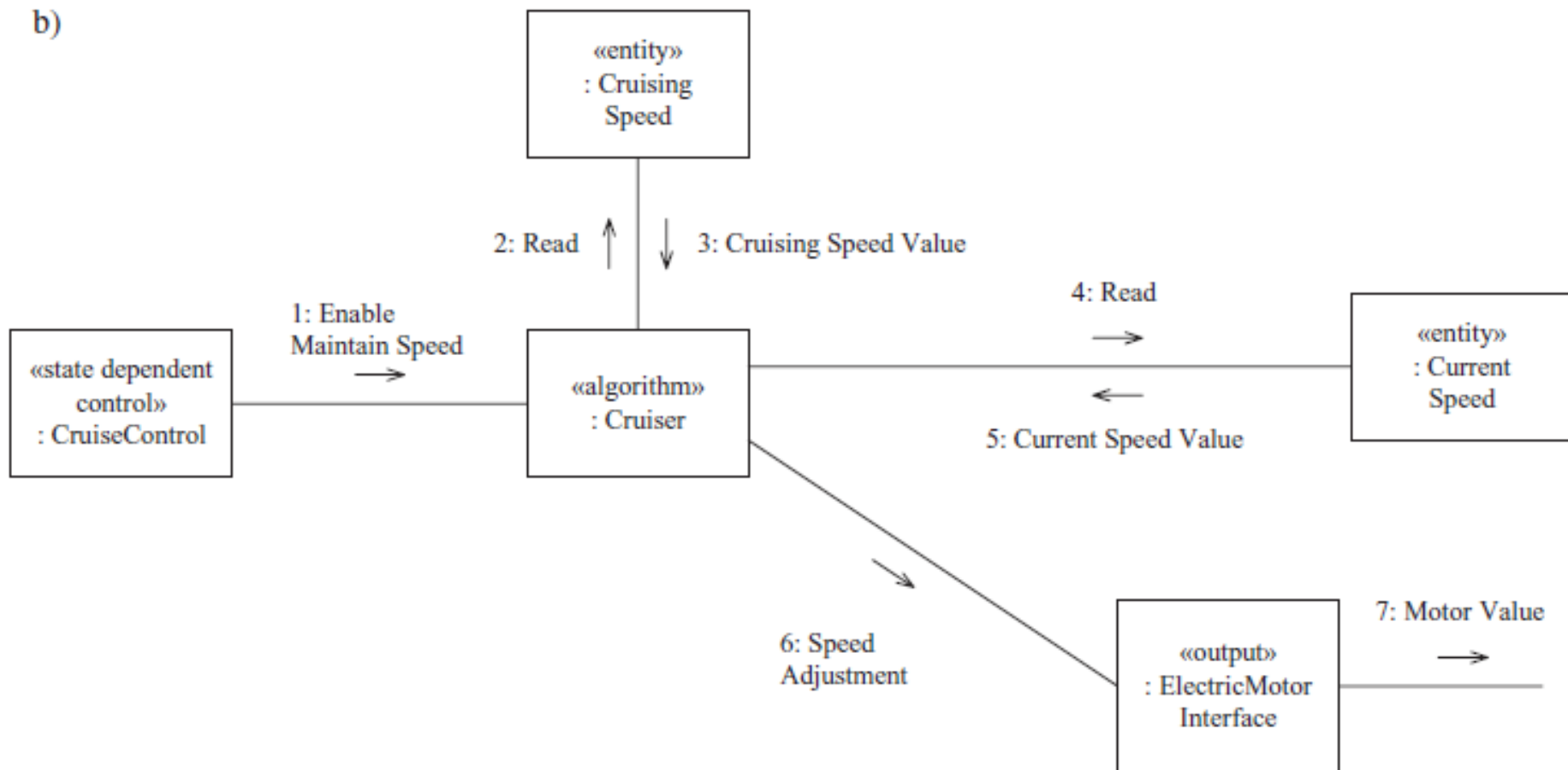
- 封装了问题域中使用的算法
- 算法独立于其他对象而变化
- 算法被迭代精化，因此它们独立于它们所操作的数据
- 算法经常封装了计算其算法所需要的数据（初始化数据、中间数据、阈值数据等）
- 算法对象必须频繁与其他对象交互，以便执行算法



对象和类的组织

- 算法对象的例子：“巡航器”算法类

b)



对象和类的组织

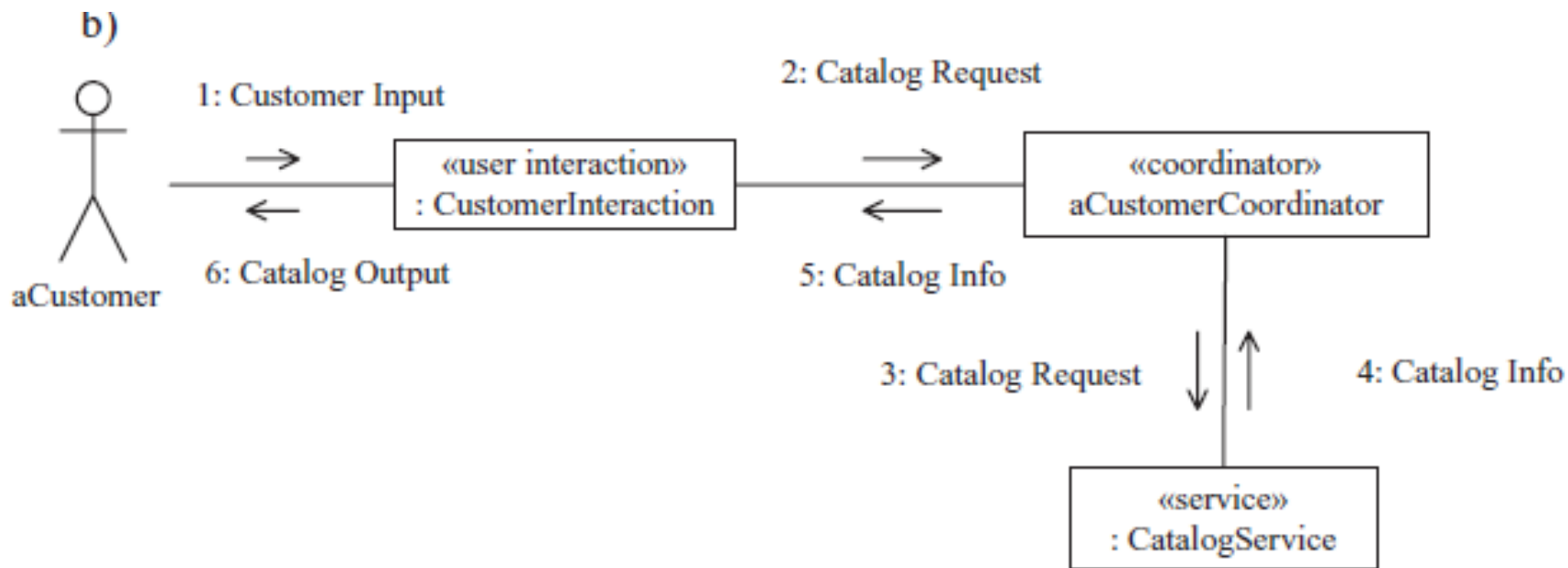
○服务对象

- 对其他对象提供服务，常用于面向服务的架构中
- 客户端从服务对象请求一个服务，服务对象做出响应
- 服务对象不会发起请求，但是为了完成服务有可能调用其他服务对象
- 可能封装了它需要用来服务客户端请求的数据，或访问其他封装了该数据的实体对象



对象和类的组织

- 服务对象的例子：“目录服务”类



总结

- 先确定问题域的静态模型
- 在动态交互建模时，将对象和类的构造标准依次用于每个用例上，从而确定每个用例中的对象
- 通过用例的交互序列来建立对象之间的交互序列





西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

THE END